BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	**Nguyễn Lương Hồng Búp**
MSSV / Lớp	**3124720009**

Mục tiêu học tập

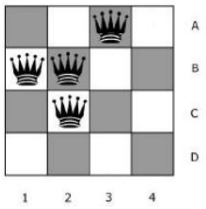
- Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).
- Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.
- Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.
- Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.
- Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
        current_state = problem.INITIAL-STATE
        loop do
                neighbor = state with lowest number of conflicts between queens obtained by
                                making one legal chess move from current_state
                if neighbor.num_conflicts >= current_state.num_conflicts
                        return current_state
                current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Ký hiệu ô theo (hàng, cột): hàng A–D từ trên xuống, cột 1–4 từ trái sang. Đặt Q1, Q2, Q3, Q4 lần lượt là các quân hậu ở A3, B1, B2, C2.

Trạng thái khởi đầu có **4 xung đột**: Q2–Q3 (cùng hàng B), Q3–Q4 (cùng cột 2), Q1–Q3 (chéo A3–B2), Q2–Q4 (chéo B1–C2).

Xét mọi nước đi hợp lệ của 4 quân, láng giềng có số xung đột thấp nhất (= 1) là duy nhất: **Q3 đi từ B2 xuống D4** (đi chéo qua C3, không bị chặn). Sau một vòng lặp:

Quân hậu	Trước	Sau 1 vòng lặp	Ghi chú
Q1	A3	A3	giữ nguyên
Q2	B1	B1	giữ nguyên
Q3	B2	**D4**	quân được di chuyển
Q4	C2	C2	giữ nguyên

Số xung đột sau vòng lặp 1 là **1** (chỉ còn cặp Q2–Q4 cùng đường chéo B1–C2).

b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

☐ A) Không dừng

☒ **B) Cực trị cục bộ (local optimum)**

☐ C) Cực trị toàn cục (global optimum)

☒ **D) Vai bằng (shoulder)**

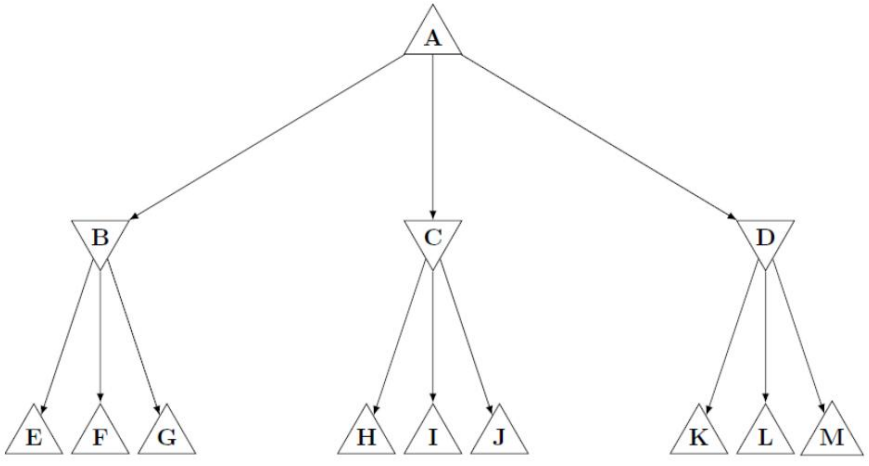
☐ E) Không thuộc các trường hợp trên

Giải thích:

- Vòng lặp 2: các láng giềng tốt nhất (ví dụ Q4: C2 → D2 hoặc C2 → C4) vẫn có 1 xung đột. Vì điều kiện dừng là neighbor.num_conflicts ≥ current_state.num_conflicts (1 ≥ 1) nên thuật toán **dừng** (không chọn A) và trả về trạng thái A3, B1, D4, C2.
- Trạng thái cuối còn 1 xung đột ≠ 0 nên **không phải cực trị toàn cục** (không chọn C). Nghiệm toàn cục của 4-Queens có 0 xung đột.
- Không có láng giềng nào tốt hơn nghiêm ngặt nên đây là **cực trị cục bộ** theo thuật toán (chọn B).
- Đồng thời nó nằm trên một vùng phẳng (các láng giềng cùng 1 xung đột) và từ vùng phẳng này vẫn có đường đi tới trạng thái 0 xung đột, nên cũng là **vai bằng (shoulder)** (chọn D). Thuật toán dừng ở đây vì dùng dấu ≥ nên không đi ngang qua vùng phẳng.

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

☐ B → C → D

☐ C → D → B

☒ **D → C → B**

Giải thích: f(B) = 4, f(C) = 14, f(D) = 19. Nút cha A là MAX nên ưu tiên con có f lớn nhất: D (19) → C (14) → B (4).

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Duyệt theo thứ tự D → C → B, tại gốc A: $\alpha = -\infty$, $\beta = +\infty$.

Cây con	**α, β khi vào**	**Diễn biến**	**Trả về**
D (MIN)	$\alpha = -\infty$, $\beta = +\infty$	Xét K = 23 (v = 23), L = 20 (v = 20), M = 26 (v = 20). Không có cắt tỉa vì $\alpha = -\infty$.	20 → A: $\alpha = 20$
C (MIN)	$\alpha = 20$, $\beta = +\infty$	Xét H = 17 ≤ $\alpha = 20$, MAX đã có lựa chọn tốt hơn nên **cắt I, J**.	≤ 17
B (MIN)	$\alpha = 20$, $\beta = +\infty$	Xét E = 5 ≤ $\alpha = 20$, nên **cắt F, G**.	≤ 5

Giá trị tại A = max(20, ≤17, ≤5) = **20**, nước đi chọn là D.

Các nút KHÔNG được thăm: I, J (dưới C) và F, G (dưới B). Các nút đã thăm: K, L, M, H, E.

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

☐ Có

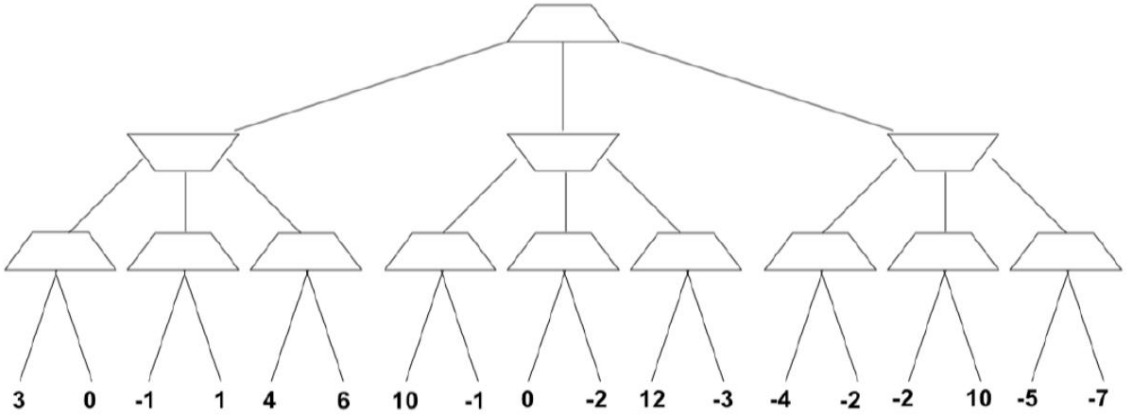
☒ **Không**

Giải thích:

- Alpha-Beta chỉ đảm bảo cùng kết quả với Minimax **ở cùng một độ sâu tìm kiếm**; thứ tự sắp xếp các nhánh (theo f) chỉ giúp cắt tỉa nhiều hơn chứ không đổi kết quả.
- Ở đây tìm kiếm bị **giới hạn độ sâu** (đến các nút cháu E…M) và dùng **hàm đánh giá f(s)** (ước lượng) thay cho giá trị thật của các nút lá. Nếu f không chính xác hoặc nếu tìm sâu hơn thì giá trị các nhánh có thể đổi thứ hạng, nên nước đi có thể khác với Minimax trên toàn bộ cây.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



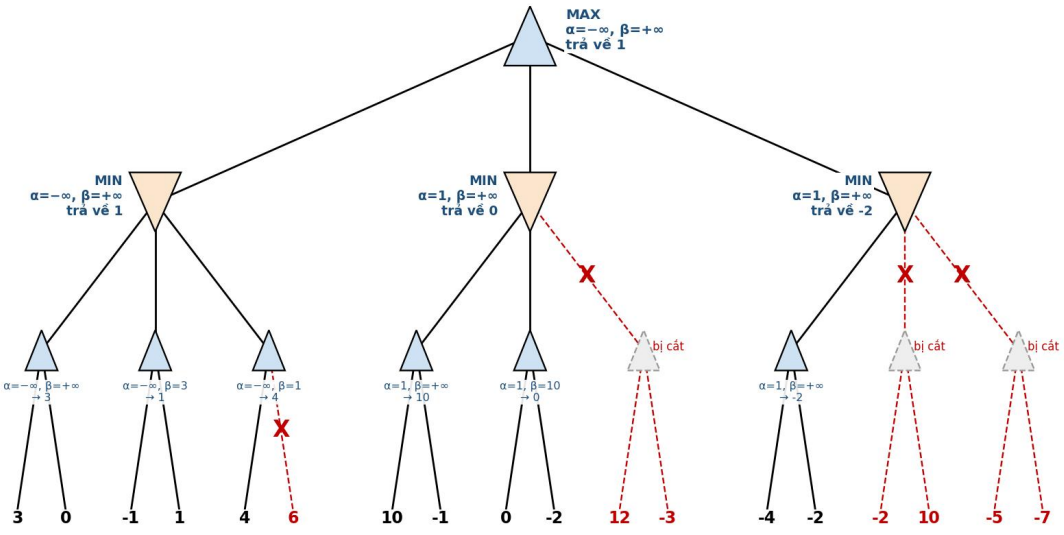
Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

- Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.
- Ghi giá trị được trả về tại mỗi nút trung gian.
- Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.
- Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Bài làm:

Cây có gốc MAX, 3 nút MIN (M1, M2, M3 từ trái sang phải), mỗi nút MIN có 3 nút MAX (Mi.1, Mi.2, Mi.3), mỗi nút MAX có 2 lá. Hình dưới ghi α, β khi vào mỗi nút, giá trị trả về (→) và đánh dấu X lên nhánh bị cắt (các lá màu đỏ không được thăm).



Cây Alpha-Beta sau khi giải (α, β: giá trị khi vào nút; → giá trị trả về; X: nhánh bị cắt)

Nút	α (vào)	β (vào)	Trả về	Ghi chú
Gốc (MAX)	−∞	+∞	1	sau M1: α = 1; M2, M3 không vượt được 1
M1 (MIN)	−∞	+∞	1	v = min(3, 1, 4) = 1; xét hết các con

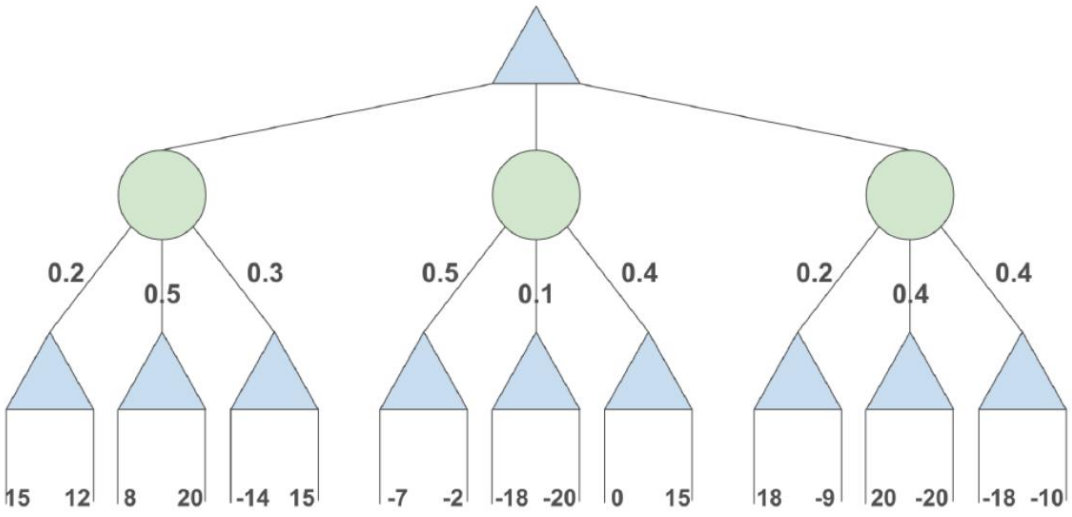
Nút	α (vào)	β (vào)	Trả về	Ghi chú
				(M1.3 trả 4 ≥ β = 1)
M1.1 (MAX)	−∞	+∞	3	lá 3, 0
M1.2 (MAX)	−∞	3	1	lá −1, 1
M1.3 (MAX)	−∞	1	4	lá 4 ≥ β = 1 nên **cắt lá 6**
M2 (MIN)	1	+∞	0	sau M2.2: v = 0 ≤ α = 1 nên **cắt M2.3**
M2.1 (MAX)	1	+∞	10	lá 10, −1; β của M2 thành 10
M2.2 (MAX)	1	10	0	lá 0, −2
M2.3 (MAX)	1	0	không thăm	**cắt**: không xét lá 12, −3
M3 (MIN)	1	+∞	−2	sau M3.1: v = −2 ≤ α = 1 nên **cắt M3.2, M3.3**
M3.1 (MAX)	1	+∞	−2	lá −4, −2
M3.2, M3.3	–	–	không thăm	**cắt**: không xét lá −2, 10, −5, −7

Các nhánh bị cắt (X): lá 6 (dưới M1.3); nút M2.3 (lá 12, −3); các nút M3.2 và M3.3 (lá −2, 10 và −5, −7), tổng cộng 7 lá không được thăm.

Giá trị Minimax tại gốc = 1 (M1 = 1, M2 = 0, M3 ≤ −2, lấy max). **Nước đi tối ưu của MAX: đi sang nhánh trái (đến M1).**

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



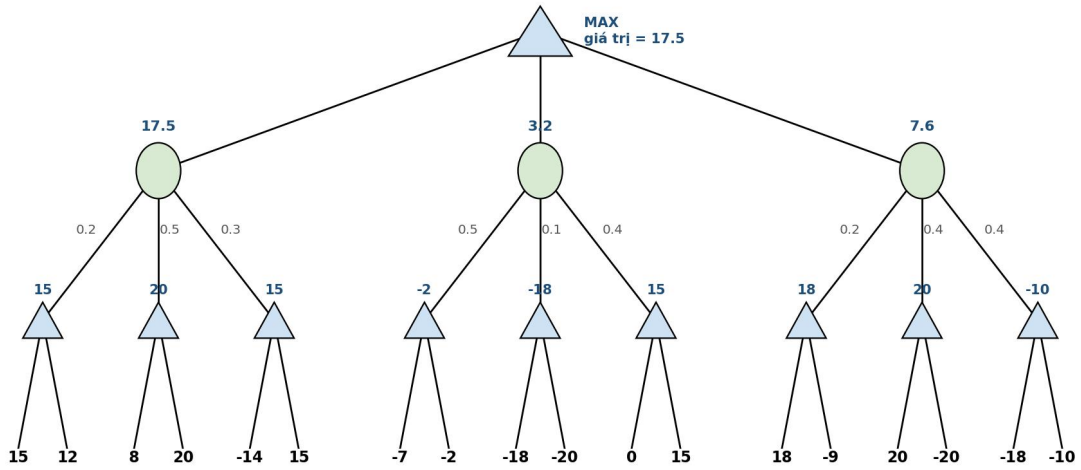
Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

Chance node	Các nút MAX (max của 2 lá)	Giá trị chance node (kỳ vọng)

Chance node	Các nút MAX (max của 2 lá)	Giá trị chance node (kỳ vọng)
Trái	max(15, 12) = 15; max(8, 20) = 20; max(−14, 15) = 15	0,2·15 + 0,5·20 + 0,3·15 = 3 + 10 + 4,5 = **17,5**
Giữa	max(−7, −2) = −2; max(−18, −20) = −18; max(0, 15) = 15	0,5·(−2) + 0,1·(−18) + 0,4·15 = −1 − 1,8 + 6 = **3,2**
Phải	max(18, −9) = 18; max(20, −20) = 20; max(−18, −10) = −10	0,2·18 + 0,4·20 + 0,4·(−10) = 3,6 + 8 − 4 = **7,6**

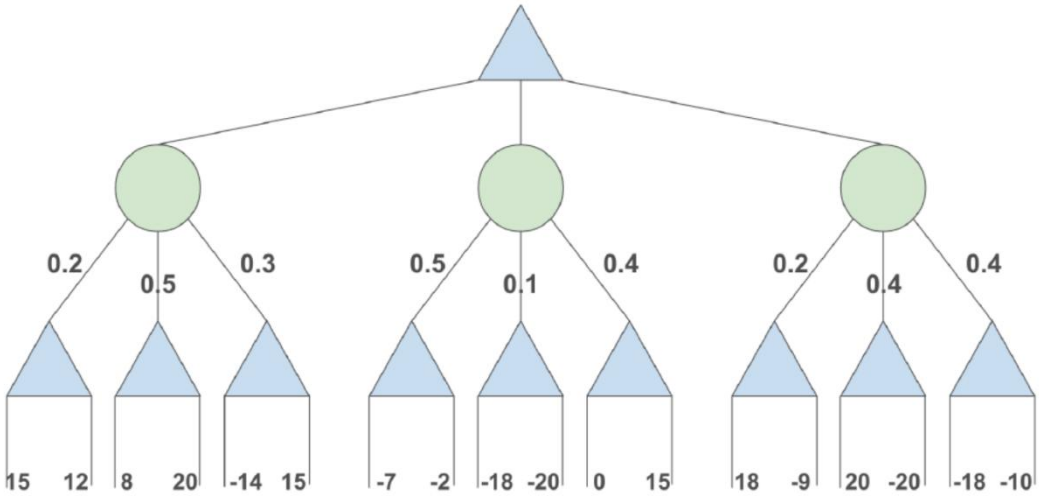
Nút gốc (MAX) = max(17,5; 3,2; 7,6) = **17,5** (chọn nhánh trái).



b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo kết quả chính xác? Giải thích.

Không có nút nào có thể cắt tỉa. Giá trị của chance node là trung bình có trọng số của tất cả các con. Khi lá chưa bị chặn giá trị, một lá chưa thăm có thể lớn hoặc nhỏ tùy ý và làm thay đổi kỳ vọng bao nhiêu cũng được, nên không thể kết luận một chance node chắc chắn không vượt qua giá trị tốt nhất hiện có. Tương tự, mỗi nút MAX cần xem cả hai lá vì lá còn lại có thể lớn hơn. Do đó phải duyệt toàn bộ 18 lá.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [−20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết; vẫn duyệt từ trái sang phải.

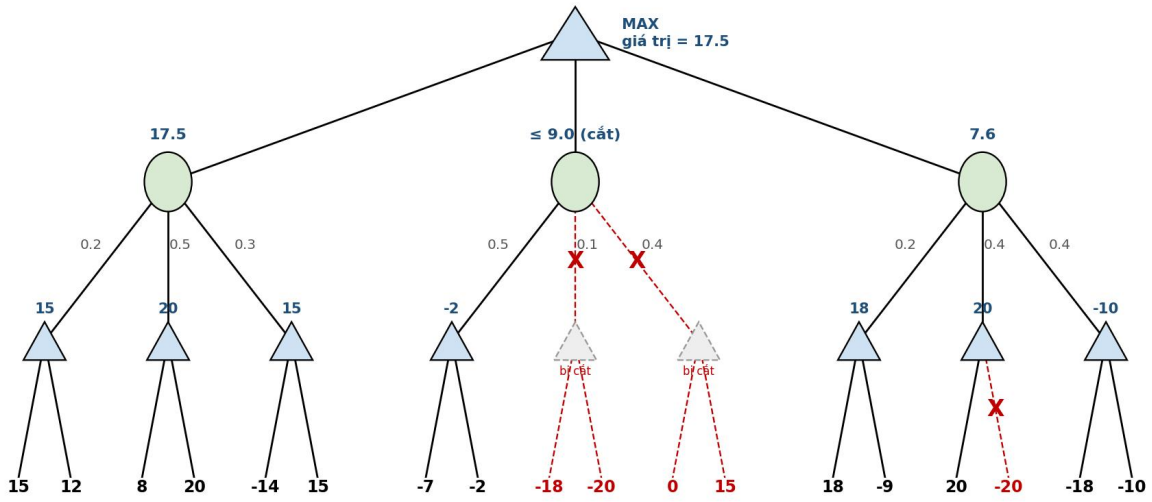


Hình 5. Cây Expectimax dùng cho câu 4(c)

Bài làm:

- **Nhánh trái:** chưa có giá trị tốt nhất nào (α của gốc = −∞) nên duyệt hết: 15, 20, 15 → 17,5. Gốc có giá trị hiện tại 17,5.
- **Nhánh giữa:** nút MAX thứ nhất = max(−7, −2) = −2 (xác suất 0,5). Cận trên của chance node: 0,5·(−2) + 0,5·20 = 9 < 17,5. Dù hai nút MAX còn lại có giá trị lớn nhất là 20 thì chance node này vẫn không vượt 17,5, nên **cắt hai nút MAX thứ 2 và thứ 3** (các lá −18, −20, 0, 15).
- **Nhánh phải:** nút MAX 1 = 18, cận trên 0,2·18 + 0,8·20 = 19,6 > 17,5 nên tiếp tục. Nút MAX 2: lá đầu 20 đã bằng giá trị lớn nhất cho phép nên MAX = 20 và **cắt lá thứ hai (−20)**; cận trên vẫn 19,6 nên tiếp tục. Nút MAX 3 = max(−18, −10) = −10. Giá trị chance node = 3,6 + 8 − 4 = 7,6.

Kết quả: gốc = max(17,5; ≤ 9; 7,6) = **17,5**, nước đi tối ưu là nhánh trái. Tổng cộng cắt **5 lá** (−18, −20, 0, 15 ở nhánh giữa và −20 ở nhánh phải).



Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Tiêu chí	Forward Checking (FC)	AC-3
Phạm vi lan truyền	Chỉ **một bước**: khi gán X = v, loại các giá trị không tương thích khỏi miền của các biến **chưa gán kề với X**. Không kiểm tra ràng buộc giữa các biến chưa gán với nhau, không lan tiếp.	Lan truyền **dây chuyền** trên toàn đồ thị: khi miền một biến bị thu hẹp thì đưa các cung liên quan vào hàng đợi và kiểm tra lại, lặp tới khi hết thay đổi. Đảm bảo nhất quán cung cho mọi cặp biến.
Thời điểm thực hiện	Sau **mỗi phép gán** trong lúc backtracking search.	Có thể chạy **trước khi tìm kiếm** (tiền xử lý) và/hoặc sau mỗi phép gán (MAC, Maintaining Arc Consistency).
Chi phí tính toán	Rẻ: mỗi phép gán chỉ kiểm tra các cung (Xj, X) của các láng giềng chưa gán, khoảng O(d) cho mỗi láng giềng.	Đắt hơn: trường hợp xấu $O(e \cdot d^3)$ với e cung, d là kích thước miền; mỗi lần gọi có thể duyệt nhiều cung.
Khả năng phát hiện thất bại	Phát hiện được khi miền một biến chưa gán rỗng, nhưng bỏ sót những xung đột gián tiếp.	Mạnh hơn: phát hiện sớm hơn và thu hẹp miền nhiều hơn, giảm số lần backtrack nhưng tốn thêm chi phí mỗi bước.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

- MRV chọn biến có **ít giá trị hợp lệ còn lại nhất** (nguyên tắc "fail-first"): biến bị ràng buộc nhiều nhất thường là nơi dễ thất bại sớm nhất.
- Vì mọi biến đều phải được gán, nên phát hiện sớm nhánh không có nghiệm (miền rỗng) giúp **cắt cả cây con lớn** và giảm số lần backtrack; biến chỉ còn 1 giá trị được gán ngay (nước đi bắt buộc).
- Cũng làm giảm hệ số nhánh hiệu dụng của cây tìm kiếm vì biến được chọn có ít lựa chọn nhất.

ii) Least Constraining Value (LCV)

- LCV chọn, trong các giá trị của biến đã chọn, giá trị **loại bỏ ít giá trị nhất khỏi miền các biến lân cận**, tức là giữ cho các biến còn lại nhiều lựa chọn nhất (nguyên tắc "fail-last").
- Với một biến, ta chỉ cần **một** giá trị thành công, nên nên thử trước giá trị có nhiều khả năng dẫn đến nghiệm. Nhờ đó tăng cơ hội tìm được nghiệm mà không phải backtrack.
- MRV dùng để chọn biến, LCV để chọn giá trị; hai heuristic bổ sung cho nhau nên thường dùng cùng nhau.

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

- Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.
- Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.
- Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.

- Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C.
- Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng.

a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

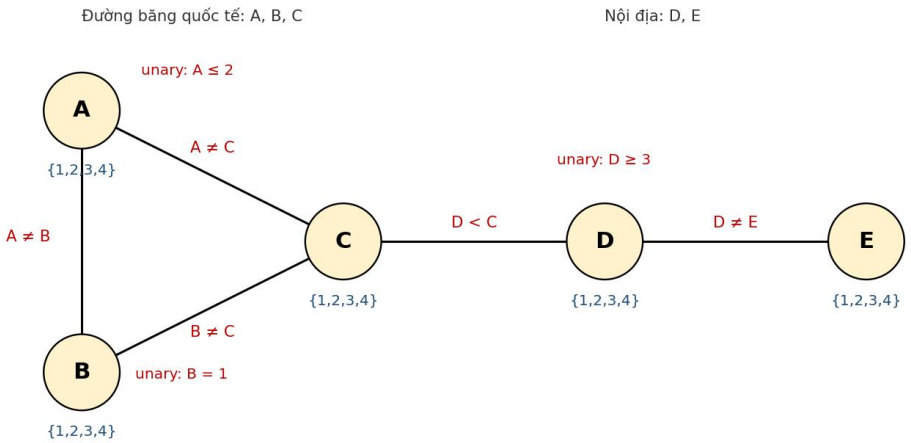
- **Biến:** với mỗi máy bay $X \in \{A, B, C, D, E\}$: khung giờ **t_X** và đường băng **r_X** (10 biến). Có thể gộp thành 5 biến X với miền là cặp (t, r).
- **Miền giá trị:** $t_X \in \{1, 2, 3, 4\}$; $r_X \in \{QT, ND\}$ (QT: quốc tế, ND: nội địa).
- **Ràng buộc đơn (unary):**
  - $t_B = 1$
  - $t_D \geq 3$
  - $t_A \leq 2$
- **Ràng buộc nhị phân (binary):**
  - $t_D < t_C$ (D hạ cánh trước khi C cất cánh)
  - Với mọi cặp $X \neq Y$: $(r_X = r_Y) \Rightarrow (t_X \neq t_Y)$, tương đương $(t_X, r_X) \neq (t_Y, r_Y)$ (10 cặp: AB, AC, AD, AE, BC, BD, BE, CD, CE, DE).

Bổ sung hai ràng buộc sau:

- A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.
- D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

- **Miền mới:** mỗi biến chỉ còn khung giờ: $Dom(A) = Dom(B) = Dom(C) = Dom(D) = Dom(E) = \{1, 2, 3, 4\}$. (Trước khi dùng unary, chưa loại giá trị nào.)
- **Ràng buộc đơn:** $B = 1$; $D \geq 3$; $A \leq 2$.
- **Ràng buộc nhị phân:** do A, B, C cùng đường băng quốc tế: $A \neq B$, $A \neq C$, $B \neq C$. D, E cùng đường băng nội địa: $D \neq E$. Cộng thêm $D < C$. Các cặp khác đường băng (ví dụ A–D, A–E, B–D, B–E, C–E) không còn ràng buộc.



Đồ thị ràng buộc: 5 cung nhị phân A–B, A–C, B–C, C–D, D–E

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộc unary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

Bước 1 – unary: A ≤ 2 → A = {1, 2}; B = 1 → B = {1}; D ≥ 3 → D = {3, 4}; C, E giữ {1, 2, 3, 4}.

Bước 2 – arc consistency (AC-3):

- (A, B): A ≠ B, B = {1} nên loại 1 khỏi A → A = {2}.
- (C, A): C ≠ A, A = {2} nên loại 2 khỏi C → C = {1, 3, 4}. (C, B): loại 1 khỏi C → C = {3, 4}.
- (D, C): D < C với C ∈ {3, 4}: D = 4 không có giá trị C lớn hơn nên loại 4 khỏi D → D = {3}.
- (C, D): C > D = 3 nên loại 3 khỏi C → C = {4}.
- (E, D): E ≠ D, D = {3} nên loại 3 khỏi E → E = {1, 2, 4}. Các cung còn lại đều nhất quán.

Miền sau cùng (gạch bỏ giá trị bị loại): A = {2}, B = {1}, C = {4}, D = {3}, E = {1, 2, 4}.

A	~~1~~	**2**	~~3~~	4
B	**1**	~~2~~	~~3~~	4
C	~~1~~	~~2~~	~~3~~	**4**
D	~~1~~	~~2~~	**3**	4
E	**1**	**2**	~~3~~	**4**

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	**1**	**2**	**3**	**4**
B	**1**	**2**	**3**	**4**
C	**1**	**2**	**3**	**4**
D	**1**	**2**	**3**	**4**
E	**1**	**2**	**3**	**4**

Miền sau khi áp dụng unary (A ≤ 2, B = 1, D ≥ 3):

A	**1**	**2**	~~3~~	4
B	**1**	~~2~~	~~3~~	4
C	**1**	**2**	**3**	**4**
D	~~1~~	~~2~~	**3**	**4**
E	**1**	**2**	**3**	**4**

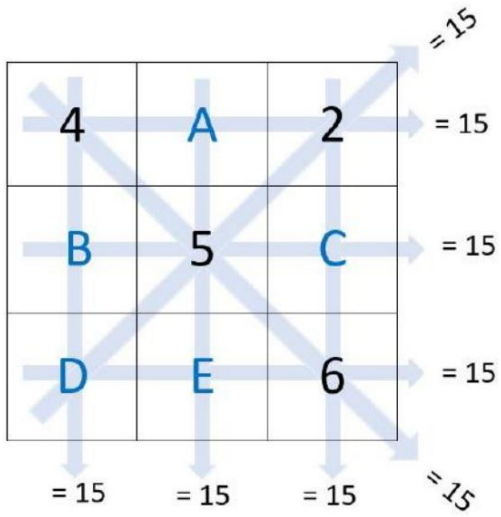
Bước	Chọn biến / giá trị	Forward Checking cập nhật miền	Kết quả
1	B (MRV: chỉ còn 1 giá trị) → **(B, 1)**	A ≠ 1 → A = {2}; C ≠ 1 → C = {2, 3, 4}	tiếp tục
2	A (MRV: 1 giá trị) → **(A, 2)**	C ≠ 2 → C = {3, 4}	tiếp tục
3	C và D cùng có 2 giá trị, hòa nên chọn chữ nhỏ hơn: C; giá trị nhỏ hơn: **(C, 3)**	D < 3 → D = {} (rỗng)	**dead end → hoàn tác (C, 3)**
4	Thử giá trị kế tiếp: **(C, 4)**	D < 4 → D = {3}	tiếp tục
5	D (MRV: 1 giá trị) → **(D, 3)**	E ≠ 3 → E = {1, 2, 4}	tiếp tục
6	E (biến cuối), giá trị nhỏ nhất → **(E, 1)**	–	đủ nghiệm

Thứ tự các cặp (biến, giá trị) phát sinh: (B, 1), (A, 2), (C, 3), (C, 3) bị hoàn tác, (C, 4), (D, 3), (E, 1).

Nghiệm: A = 2, B = 1, C = 4, D = 3, E = 1. Kiểm tra: A, B, C khác nhau (2, 1, 4) trên đường băng quốc tế; D = 3 ≠ E = 1 trên đường băng nội địa; D = 3 < C = 4.

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô được cho trước, các ô ký hiệu chữ cái cần được xác định.

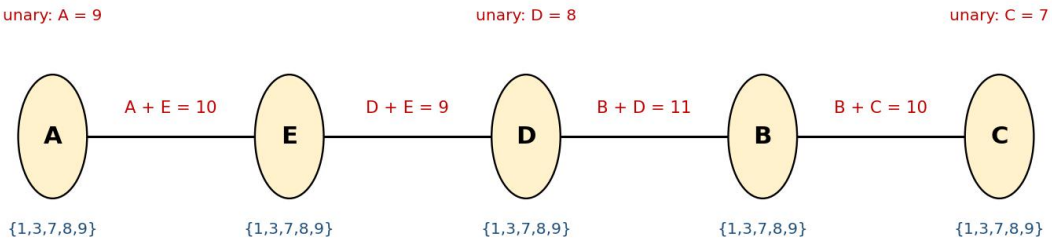


Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

- **Biến:** A, B, C, D, E (các ô còn trống).
- **Miền:** các số từ 1 đến 9 chưa xuất hiện ở các ô đã cho (2, 4, 5, 6): Dom = {1, 3, 7, 8, 9} cho mỗi biến.
- **Ràng buộc alldiff:** A, B, C, D, E đôi một khác nhau (và khác 2, 4, 5, 6, đã được loại khỏi miền).
- **Ràng buộc đơn (unary)** (các đường chỉ có một ô trống):
  - Hàng 1: 4 + A + 2 = 15 ⇒ A = 9
  - Cột 3: 2 + C + 6 = 15 ⇒ C = 7
  - Đường chéo phụ: 2 + 5 + D = 15 ⇒ D = 8
- **Ràng buộc nhị phân** (các đường có hai ô trống):
  - Hàng 2: B + 5 + C = 15 ⇒ B + C = 10
  - Hàng 3: D + E + 6 = 15 ⇒ D + E = 9
  - Cột 1: 4 + B + D = 15 ⇒ B + D = 11
  - Cột 2: A + 5 + E = 15 ⇒ A + E = 10
  - Đường chéo chính: 4 + 5 + 6 = 15 đã thỏa, không chứa biến nên không tạo ràng buộc.

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.



Đồ thị ràng buộc nhị phân: A – E – D – B – C (dạng đường thẳng)

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

Khởi tạo: Dom(A) = Dom(B) = Dom(C) = Dom(D) = Dom(E) = {1, 3, 7, 8, 9}. Áp dụng unary: A = {9}, C = {7}, D = {8}. Hàng đợi chứa các cung: (B, C), (C, B), (B, D), (D, B), (D, E), (E, D), (A, E), (E, A).

Cung xét	Lập luận	Miền sau khi cập nhật
(B, C): B + C = 10	C = 7 ⇒ B = 3; loại 1, 7, 8, 9 khỏi B	B = {3}
(B, D): B + D = 11	D = 8 ⇒ B = 3, khớp	B = {3} (không đổi)
(E, D): D + E = 9	D = 8 ⇒ E = 1; loại 3, 7, 8, 9 khỏi E	E = {1}

Cung xét	Lập luận	Miền sau khi cập nhật
(E, A): A + E = 10	A = 9 ⇒ E = 1, khớp	E = {1} (không đổi)
Các cung ngược (C, B), (D, B), (D, E), (A, E)	C = 7 ↔ B = 3; D = 8 ↔ B = 3, E = 1; A = 9 ↔ E = 1: đều nhất quán	không đổi

Hàng đợi rỗng. Mọi miền đều còn đúng một giá trị: **A = 9, B = 3, C = 7, D = 8, E = 1**, nên AC-3 giải được bài toán mà không cần tìm kiếm. Alldiff thỏa: {9, 3, 7, 8, 1} đôi một khác nhau và khác 2, 4, 5, 6.

Nghiệm (Magic Square):

4	**9**	**2**
3	**5**	**7**
8	**1**	**6**

Kiểm tra: các hàng 4+9+2 = 3+5+7 = 8+1+6 = 15; các cột 4+3+8 = 9+5+1 = 2+7+6 = 15; hai đường chéo 4+5+6 = 2+5+8 = 15.

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

- AJ = {Đỏ, Vàng, Xanh lá}
- Sunny = {Đỏ, Vàng, Hồng}
- Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại thứ tự gán.

Bước	Chọn biến (MRV)	Chọn giá trị (LCV)	Gán
1	Ellyse có 2 giá trị (AJ, Sunny có 3 giá trị) nên chọn Ellyse	Đỏ loại Đỏ khỏi AJ và Sunny (loại 2 giá trị); Hồng chỉ loại Hồng khỏi Sunny (loại 1 giá trị) nên chọn Hồng	Ellyse = Hồng
2	AJ = {Đỏ, Vàng, Xanh lá} (3); Sunny = {Đỏ, Vàng} (2) nên chọn Sunny	Đỏ loại Đỏ khỏi AJ (1); Vàng loại Vàng khỏi AJ (1): hòa, theo thứ tự cầu vồng Đỏ trước Vàng	Sunny = Đỏ
3	Chỉ còn AJ = {Vàng, Xanh lá}	Không còn biến lân cận chưa gán: hòa, theo thứ tự cầu vồng Vàng trước Xanh lá	AJ = Vàng

Thứ tự gán: Ellyse = Hồng → Sunny = Đỏ → AJ = Vàng. **Số lần backtrack = 0.**

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

Bước	Chọn biến (Max Remaining)	Chọn giá trị (Most Constraining)	Gán
1	AJ và Sunny cùng có 3 giá trị (nhiều nhất) nên hòa; chọn theo thứ tự đề	Đỏ loại Đỏ khỏi Sunny và Ellyse (2); Vàng loại Vàng khỏi Sunny (1); Xanh lá loại 0:	AJ = Đỏ

Bước	Chọn biến (Max Remaining)	Chọn giá trị (Most Constraining)	Gán
	bài: AJ	chọn Đỏ	
2	Sunny = {Vàng, Hồng} (2) nhiều hơn Ellyse = {Hồng} (1) nên chọn Sunny	Hồng loại Hồng khỏi Ellyse (làm Ellyse rỗng, 1 giá trị); Vàng loại 0: chọn Hồng	Sunny = Hồng
3	Ellyse = {} rỗng: **dead end**	–	**backtrack lần 1**: hoàn tác Sunny = Hồng
4	Thử giá trị còn lại của Sunny	–	Sunny = Vàng
5	Ellyse = {Hồng}	–	Ellyse = Hồng

Số lần backtrack = 1 (so với 0 ở câu a). Nghiệm cuối: AJ = Đỏ, Sunny = Vàng, Ellyse = Hồng.

So sánh và giải thích:

- Câu (a): MRV chọn **biến khó nhất** (Ellyse chỉ còn 2 màu) trước, LCV chọn giá trị **ít làm hẹp** các biến còn lại (Hồng) nên giữ lại nhiều lựa chọn và tìm được nghiệm ngay, không phải backtrack.
- Câu (b): các heuristic ngược lại chọn biến **dễ nhất** trước (AJ), rồi chọn giá trị **làm hẹp nhiều nhất** (Đỏ, rồi Hồng của Sunny làm Ellyse hết màu), nên dẫn tới dead end muộn và phải backtrack.
- Trong bài nhỏ này chênh lệch chỉ là 1 lần backtrack, nhưng với bài lớn, MRV + LCV giảm đáng kể không gian tìm kiếm so với thứ tự ngược lại.
- Lưu ý: ở bước 1 của câu (b), việc hòa giữa AJ và Sunny được phá hòa theo thứ tự trong đề (AJ trước). Nếu chọn Sunny trước thì có thể ra 0 lần backtrack, vì kết quả phụ thuộc cách phá hòa.

--- Hết ---